RecruitO · Learning Series (v2)

FastAPI, auth, password reset, services & LiveKit

The Python backend layer by layer, updated for today: the password-reset flow, AssemblyAI transcription, the LiveKit token endpoint, manager-driven scheduling, and the interview WebSockets (including the stale-socket fix).

Updated study guide · reflects the project as of 27 October 2026

1. The layers

Layer (folder)	Responsibility
main.py	Creates the app, registers routers, sets CORS (now largely redundant — same origin), starts the worker, /health.
routers/	HTTP endpoints: parse, check permission, call services, return a response.
dependencies.py	get_current_user (reads cookie) + require_roles (checks role).
schemas/	Pydantic request/response shapes.
services/	Business logic: email, screening, scheduling, transcription, LiveKit tokens, biometrics.
graphs/	LangGraph AI workflows.
models/ + db/ + migrations/	ORM tables, the DB connection, and versioned schema changes.

2. Authentication & the new password reset

Login signs a JWT (holding user id + role) and stores it in an HttpOnly cookie; every protected request is guarded by `require_roles(...)`. Sessions now last **8 hours** (`ACCESS_TOKEN_EXPIRE_MINUTES=480`).

2.1 The guard (dependencies.py)

```
def get_current_user(request):
    token = request.cookies.get("access_token")
    if not token: raise HTTPException(401, "Not authenticated")
    payload = decode_access_token(token)        # verify signature + expiry
    user = <load active User by payload["sub"]>
    return user

def require_roles(*allowed):                     # e.g. require_roles("hiring_manager")
    def checker(user = Depends(get_current_user)):
        if user.role not in allowed: raise HTTPException(403, "no permission")
        return user
    return checker
```

401 means 'who are you?' (no/expired cookie); 403 means 'not allowed'. The frontend turns a mid-session 401 into an automatic redirect to sign in.

2.2 Forgot / reset password (NEW)

The flow
1) User submits their email to `POST /auth/forgot-password`. 2) If an active account exists, the server stores a **hashed, 1-hour, single-use token** and emails a link (`/reset-password.html?token=...`) via SMTP. 3) The user opens the link and submits a new password to `POST /auth/reset-password`, which verifies the token, updates the password hash, and marks the token used. The forgot endpoint always returns the same message, so it never reveals which emails exist.

```
@router.post("/forgot-password")
def forgot_password(request):
    user = <active user with this email or None>
    if user is None: return generic_message        # do not reveal existence
    delete old PasswordResetToken rows for user
    raw = secrets.token_urlsafe(32)                 # emailed to the user
    save PasswordResetToken(token_hash=sha256(raw), user_id, expires_at=now+1h)
    send_password_reset_email(user.email, f".../reset-password.html?token={raw}")
    return generic_message

@router.post("/reset-password")
def reset_password(request):
    row = <PasswordResetToken with sha256(request.token)>
    if row is None or row.used_at or row.expires_at <= now: raise HTTPException(400, ...)
    user.password_hash = hash_password(request.new_password)
    row.used_at = now                               # single use
```

The raw token is **never stored** — only its SHA-256 hash, exactly like the interview join tokens. A new table `password_reset_tokens` was added via an Alembic migration.

3. Services

Service	What it does
interview_scheduling_queue.py	When a CV's latest score exceeds 65/100, flips the application to 'selected for interview' and EMAILS the hiring manager to schedule (replaced Google-Calendar auto-scheduling).
notifications.py	Queues emails; the worker sends them via SMTP (prefers SMTP_*; APP_MAIL_* is a fallback).
transcription.py	NEW provider: transcodes each audio clip to WAV with ffmpeg, then calls AssemblyAI (upload -> transcript -> poll), forced to English.
livekit_tokens.py	NEW: mints a LiveKit Cloud join token (API key/secret) so a participant can join the room; media relays through LiveKit.
screening_queue.py / assessment / interview_analysis	CV scoring and interview analysis via Gemini + LangGraph.
face_biometrics.py	Optional local OpenCV face match for the identity check.

3.1 Transcription: ffmpeg + AssemblyAI (line by line)

```
async def transcribe_audio_chunk(audio, mime_type):
    key = os.getenv("ASSEMBLYAI_API_KEY")
    data, _ = await _to_wav(audio, mime_type)        # ffmpeg: webm -> 16kHz mono WAV
    async with httpx.AsyncClient() as client:
        up = await client.post(".../v2/upload", headers={"authorization":key}, content=data)
        tid = (await client.post(".../v2/transcript",
               json={"audio_url": up.json()["upload_url"], "language_code":"en"})).json()["id"]
        for _ in range(40):                          # poll until the short clip is done
            body = (await client.get(f".../v2/transcript/{tid}")).json()
            if body["status"] == "completed": return body["text"]
            if body["status"] == "error": raise RuntimeError(...)
            await asyncio.sleep(1)
```

Line	What it does
`_to_wav`	Runs ffmpeg to convert the browser's webm/opus clip into 16 kHz mono WAV — AssemblyAI can't decode webm directly. This was the key fix that made transcription work.
`/v2/upload`	Uploads the audio bytes; AssemblyAI returns a private URL for them.
`/v2/transcript`	Requests a transcript of that upload; language_code='en' forces English (no per-clip auto-detect).
`poll loop`	Short clips finish in a few seconds; we poll until 'completed' and return the text (empty if there was no speech).

Where the audio comes from

Each participant's browser records ~3.5-second self-contained clips of ONLY the microphone track and streams them over a WebSocket to the backend; the backend transcribes each clip and broadcasts the text, tagged with the speaker's role.

3.2 LiveKit token endpoint (NEW)

```
@interview_access_router.post("/{interview_id}/livekit-token")
def create_livekit_token(interview_id, current_user = Depends(require_roles("candidate","interviewer"))):
    if not livekit_tokens.is_configured(): raise HTTPException(503, "LIVEKIT_NOT_CONFIGURED")
    <verify the user is this interview's candidate or interviewer, and the join window is open>
    room = f"interview-{interview_id}"
    token = livekit_tokens.mint_token(identity=..., name=..., room=room)
    return {"url": LIVEKIT_URL, "token": token, "room": room}
```

The backend only signs a short-lived join token with the LiveKit API key/secret; it never relays media. If LiveKit isn't configured, the room falls back to peer-to-peer (same-network only).

4. Manager-driven scheduling (a real endpoint)

`PUT /applications/{id}/interview` is how the manager books an interview after being emailed that a candidate is eligible. Standard shape: guard → validate → load → mutate → queue emails → commit.

```
def schedule_interview(application_id, request, _user = Depends(require_roles("hiring_manager"))):
    interviewer = <active interviewer by request.interviewer_id> or 422
    interview = <existing for application> or new Interview(application_id=...)
    interview.scheduled_at = request.scheduled_at (UTC); interview.duration_minutes = ...
    interview.interviewer_id = interviewer.id; interview.status = "scheduled"
    interview.token_hash = sha256(secrets.token_urlsafe(32))   # join link secret
    session.flush()                              # insert AFTER required fields are set
    queue_email(candidate ... join_url); queue_email(interviewer ...)
    session.commit()
```

The `flush()` runs only after the required columns are set — flushing earlier inserted a row with a null scheduled_at (a NOT-NULL violation), which the browser saw as 'Could not reach the API'.

5. Interview WebSockets (presence, media signalling, transcript)

- **/interviews/{id}/signal** — presence (who has joined), the 'end' event, the optional identity frame, and peer-to-peer offer/answer/ice when LiveKit is off.
- **/interviews/{id}/transcript** — receives the consent message, replies 'transcription_ready', then receives audio chunks, transcribes each, and broadcasts the text to both participants.

> **The stale-socket fix**
> Each WebSocket room keeps one slot per role in memory. Previously a new connection for an already-filled role was rejected with code 4409 — so if a tab closed abruptly, the slot stayed stuck and transcription was permanently broken until the API restarted. Now a new connection **evicts the old socket** and takes over, and cleanup only removes its own slot. This was the real cause of 'transcription doesn't work'.